

ESE 218 – Project 2

Lasya Maganti
McKelvey School of Engineering
Washington University in St. Louis
Email: l.maganti@wustl.edu

I. INTRODUCTION

The goal of this project was to apply least-squares methods to the problem of image classification using the MNIST handwritten digit dataset. Specifically, I developed a binary classifier to distinguish between images of the digit “0” and all other digits and constructed the feature matrix and label vector that corresponded with the dataset. Using least-squares optimization, I solved for the model parameters and evaluated the classifier’s accuracy, false positive rate, and false negative rate. I then examined how performance changed when the training set was reduced, and experimented with new feature representations created through random nonlinear transformations. This project highlights the connection between linear algebra and machine learning, showing how matrix least-squares techniques can be used to analyze and classify image data.

II. BACKGROUND

Image classification problems can be framed in a way where each image is represented by a set of numerical features and assigned a known label. The goal is to learn a rule that predicts the correct label for new images. This rule can be modeled as a linear combination of features.

The least-squares classifier uses this idea to find a weight vector θ that best maps feature vectors x_i to their corresponding labels y_i by minimizing the total squared error:

$$\min_{\theta} \|A\theta - y\|_2^2,$$

where A is the feature matrix whose rows contain the features of each training image and y is the vector of class labels. Once θ is found, new images are classified according to the sign of the linear score

$$\hat{f}(x) = \text{sign}(\theta^\top x).$$

In this project, each MNIST image was represented as a vector of pixel intensities, and the classification was simplified to a binary values: separating the digit “0” (class +1) from digits “1–9” (class –1). Because many pixels are always blank, only those that were nonzero in at least 600 training images were kept as part of the feature set, forming a reduced feature matrix $A \in R^{N \times M_0}$.

To explore the effect of feature transformations, the experiment was continued by applying a random nonlinear mapping of the form

$$\tilde{x}_i = \max\{Rx_i, 0\},$$

where R is a random matrix with entries in $\{+1, -1\}$. This transformation combines and sorts out the original pixel features, introducing nonlinearity. Varying the number of random features M reveals how feature dimensionality influences classifier accuracy and generalization.

III. METHODS

A. Constructing the Feature Matrix and Label Vector

Each image in the MNIST dataset was represented as a 20×20 grid of pixel intensities. The first 5000 training images and their corresponding labels were loaded using the provided MATLAB function. To reduce dimensionality, only pixels that were nonzero in at least 600 of the training images were kept as features. A_{ij} denotes the intensity of pixel j in image i . The number of selected features was stored in M_0 .

For each training image i , the two-dimensional pixel array was flattened into a column vector $x_i \in R^{400}$, and the entries corresponding to the features were extracted to form a feature vector of length M_0 . Stacking these row vectors produced the feature matrix

$$A = \begin{bmatrix} x_1^\top \\ x_2^\top \\ \vdots \\ x_N^\top \end{bmatrix} \in R^{N \times M_0},$$

where $N = 5000$.

The label vector $y \in R^N$ was constructed according to

$$y_i = \begin{cases} +1, & \text{if the image label is 0,} \\ -1, & \text{if the image label is 1–9.} \end{cases}$$

This converts the complicated MNIST problem into a simple binary classification between the digit “0” and all other digits.

B. Solving for the Classifier Parameters

The classifier is based on a linear decision function of the form

$$\hat{f}(x) = \text{sign}(\theta^\top x),$$

where $\theta \in R^{M_0}$ is the parameter vector learned by minimizing the squared error between the predictions and true labels found from the training data:

$$\min_{\theta} \|A\theta - y\|_2^2.$$

This least-squares problem was solved in MATLAB using the backslash operator,

$$\text{theta} = A \backslash y.$$

The resulting parameter vector θ contains one coefficient per selected pixel, showing the importance of each feature for distinguishing zeros from other digits.

To visualize feature importance, the entries of θ were mapped back to their original pixel locations to form a 20×20 image.

C. Testing and Performance Metrics

The first 5000 test images and labels were loaded in the same way as the training data. For each test image $\hat{x}_i$, the same feature vector was used to identify valuable pixel information. The classifier then produced a prediction

$$\hat{y}_i = \text{sign}(\theta^\top \hat{x}_i).$$

The true label $\hat{y}_i^{\text{true}}$ was encoded using the same binary convention as the training labels. Three key performance metrics were computed:

$$\begin{aligned} \text{Error rate} &= \frac{\#\{\hat{y}_i \neq \hat{y}_i^{\text{true}}\}}{R}, \\ \text{False positive rate} &= \frac{\#\{\hat{y}_i = +1, \hat{y}_i^{\text{true}} = -1\}}{\#\{\hat{y}_i^{\text{true}} = -1\}}, \\ \text{False negative rate} &= \frac{\#\{\hat{y}_i = -1, \hat{y}_i^{\text{true}} = +1\}}{\#\{\hat{y}_i^{\text{true}} = +1\}}, \end{aligned}$$

where $R = 5000$ is the number of test samples. These describe overall accuracy, tendency to misclassify zeros as non-zero and tendency to misclassify non-zeros as zeros.

D. Reduced Training Set

To examine how training data size affects performance of the classifier, the least-squares model was retrained using only the first 100 training images. The same feature indices were used, making sure that any changes in performance were solely to reduced data rather than a different feature set. A smaller feature matrix $A_{\text{small}} \in \mathbb{R}^{100 \times M_0}$ and label vector y_{small} were constructed as before, and the parameters were obtained from

$$\text{theta_small} = A_{\text{small}} \backslash y_{\text{small}}.$$

The classifier's error, false positive, and false negative rates were then recomputed on the same 5000-image test set.

E. Random Feature Transformation

To explore the impact of feature representations, a random nonlinear feature mapping was introduced. Letting $R \in \mathbb{R}^{M \times M_0}$ be a randomly generated matrix with entries in $\{+1, -1\}$. For each training image, the new feature vector was computed as

$$\tilde{x}_i = \max\{Rx_i, 0\},$$

which first applies a random linear combination of features, then sets negative entries to zero. Stacking these $\tilde{x}_i$ vectors forms the transformed feature matrix $A_{\text{rand}} \in \mathbb{R}^{N \times M}$, from which a new least-squares solution

$$\text{theta_rand} = A_{\text{rand}} \backslash y$$

was obtained.

The process was repeated for different feature dimensions $M \in \{20, 50, 1000, 5000, 10000\}$ to analyze how the number of random features influences accuracy. Each model was evaluated on the same 5000 test images using identical transformations.

IV. RESULTS

A. Parameter Visualization

After constructing the feature matrix A and label vector y , the least-squares solution

$$\theta = (A^T A)^{-1} A^T y$$

was computed using MATLAB's backslash operator. Figure 1 visualizes the entries of θ mapped back to their corresponding pixel locations in the 20×20 image grid.

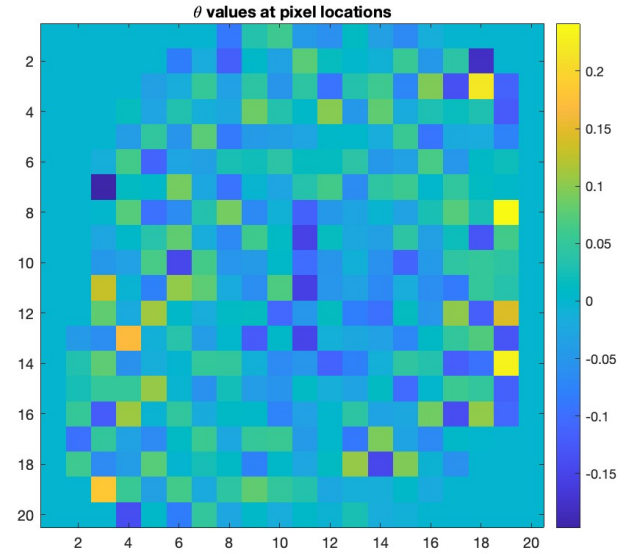


Fig. 1. Entries of parameter vector θ mapped back to 20×20 image.

Bright regions in this image correspond to pixel positions that contribute positively to predicting the digit “0,” while dark regions contribute negatively. The non-zero weights form a pattern consistent with the shape of the digit “0.” Pixels near the center, which are usually blank in zeros, tend to have negative weights, while pixels near the outer strokes have positive weights.

B. Classifier Performance

The classifier trained on 5000 images was evaluated on 5000 test images. Table I summarizes the error rate, false positive rate (predicting “0” when the digit was not zero), and false negative rate (failing to detect a true zero).

TABLE I
PERFORMANCE METRICS FOR DIFFERENT TRAINING CONDITIONS.

Training Set	Error Rate	False Positive Rate	False Negative Rate
Full (5000)	0.0194	0.0115	0.0978
Reduced (100)	0.2272	0.2064	0.4326

The full training set achieved high accuracy with less false positive than false negative rates. When the training data was reduced to only 100 images, all three performance metrics rates increased noticeably, showing much more inaccurate predictions due to poor generalizations.

C. Effect of Random Feature Transformations

To explore the effect of feature transformations, the random nonlinear mapping

$$\tilde{x}_i = \max\{Rx_i, 0\}$$

was applied with feature sizes $M = \{20, 50, 1000, 5000, 10000\}$. Each model was retrained using the transformed features and evaluated on the same test set. Table II summarizes the error rate, false positive rate (predicting “0” when the digit was not zero), and false negative rate (failing to detect a true zero) for the different values of M

TABLE II
PERFORMANCE METRICS FOR DIFFERENT FEATURE SIZES.

Feature Size M	Error Rate	False Positive Rate	False Negative Rate
20	0.0814	0.0130	0.7565
50	0.0378	0.0070	0.3413
1000	0.0102	0.0026	0.0848
5000	0.4286	0.4233	0.4804
10000	0.2980	0.2903	0.3739

As M increased, the error rate initially decreased, indicating that higher-dimensional random features captured more useful structure in the data. Beyond $M = 1000$, error rates increased drastically and then began decreasing again, suggesting overfitting may negatively affect performance metrics.

V. CONCLUSIONS

This project demonstrated how least-squares methods can be applied to the problem of image classification, connecting linear algebra concepts with basic machine learning techniques. By training on the MNIST dataset, I implemented a linear classifier that distinguishes the digit “0” from other digits using a compact set of pixel-based features. The experiments showed that:

- The least-squares classifier achieved high accuracy on the test set using the full 5000-image training data, and the visualization of θ revealed which pixel regions were most influential in distinguishing zeros.
- Reducing the training set to 100 images increased the error rate, illustrating that limited data leads to poorer generalization and less stable parameter estimates.
- Introducing random nonlinear features through $\max\{Rx_i, 0\}$ affected performance depending on feature size M : smaller M reduced accuracy, while larger M improved it up to a saturation point.

These results highlight the balance between model simplicity, data quantity, and feature representation. The least-squares framework provided a baseline for image classification, and similar techniques could be developed to multi-class problems or feature selection.

VI. REFERENCES

- [1] S. Boyd and L. Vandenberghe, Introduction to applied linear algebra: vectors, matrices, and least squares. Cambridge university press, 2018
- [2] Y. LeCun, “The MNIST dataset of handwritten digits.”
- [3] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- [4] G. Strang, *Linear Algebra and Learning from Data*, Wellesley–Cambridge Press, 2019.